pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

Document Tools

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: documents as tools

Six tools, every text with its source line

`statement_dates`

`statement_text`

`list_sections`

`read_section`

`eightk_section`

`search_indenture`

Today

1. Documents as tools
2. The body, not the contents
3. Search as a tool
4. Quotes and citations, checked
5. NOT IN TEXT, through a tool
6. An 8-K section, described
7. What a long result costs

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for documents

An FOMC statement: what it states, never a policy view

Allowed names	Never called
the statement states	hawkish or dovish
a sentence in one and not the other	a policy view
the date of the statement	what the Committee will do

An indenture: what it states, never a judgement of the issuer

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

How document tools work

The problem	A document is too long to hand a model whole, and a long result is resent at every later request
The closure	Five functions over the documents, each returning a slice
The call	A date, a section number, or an item code, and max_chars: how much text to return
You control	The documents, the tools offered, and max_chars
What comes out	A dict whose text ends with its source line: a quote always carries its document

Documents as tools

statement_text("2026-09-16", 300): the text, the cut marker, the source line

```
The Committee decided to raise the target range for the federal funds rate by 1/4 percentage po...

... [cut: 843 characters in all]
```

Source: FOMC post-meeting statement of 2026-09-16, Board of Governors of the Federal Reserve System, https:...

```
A date with no statement: {'error': 'no statement dated 2026-09-17 in the file'}
```

a dict

the same shape from every tool

max_chars

the model chooses how much text

Source:

the last line of every text

A quote the model copies always arrives with the name of its document.

Every setting of the five document tools

Tool	Setting	Type	Default
statement_dates	year	integer	required
statement_text	day	string	required
statement_text	max_chars	integer	required
list_sections	indenture	string	required
read_section	indenture	string	required
read_section	number	string	required
read_section	max_chars	integer	required
eightk_section	accession	string	required
eightk_section	item	string	required
eightk_section	max_chars	integer	required

10 settings, none with a default. Tune first: max_chars. A request below 100 is refused, in words.

How read_section works

The problem	A section number appears twice: in the table of contents and in the body
The headings	find_sections keeps each number's last heading, so the read starts at the body
The read	From that heading to the next, cut at max_chars with a marker, ended with the source line
You control	The indenture, the number, and max_chars
What comes out	The section's title and its words as the indenture wrote them

The body, not the table of contents

Every heading textkit's pattern reads as Section 4.12 in Boyd's text

```
character   1,996:  Section 4.12. Limitation on Indebtedness 56

character 198,454:  Section 4.12. Limitation on Indebtedness.
```

find_sections keeps the heading at 198,454, the last one, of 107 sections

56

a page number: the contents line

11,868

characters in the body of 4.12

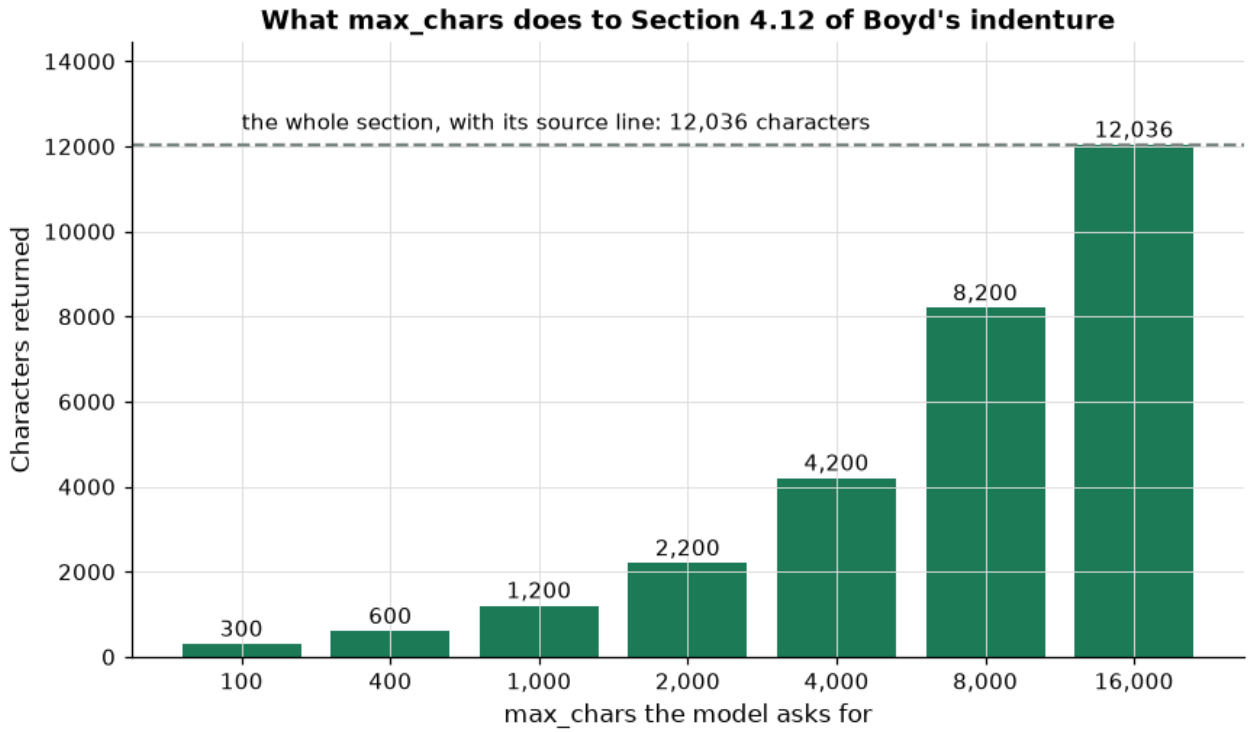
2.0 to 1.0

the ratio, in the body's first sentence

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 4.12.

read_section reads from the body's heading: the section's words, not its page number.

What max_chars does



6 of 7 limits cut the section; from 16,000 the whole section comes back. Ask for the least that holds the sentence you will quote.

How the search tool works

The problem	A question that names no section has no address to look up
The index	Boyd's chunks, each turned by MiniLM into 384 numbers, once
The search	Embed the question, dot product with every chunk, return the k highest in order
You control	k, the indenture, and the chunk size
What comes out	Ranked texts with their sections and no scores: a score's last digits move with threads

Search as a tool: ranks, never scores

search_indenture("limitation on liens", 3) over 454 chunks of 384 numbers

```
rank 1  Section 4.14   Section 4.14. Limitation on Liens. The Company shall not, and shall no
rank 2  Section 4.14   Liens (or any portion thereof) described in the definition of "Permitt
rank 3  Section 1.01   arrangements for the sale of goods entered into by that Person or any
```

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 4.14.

no score

last digits move with threads

in the key

a result is part of every later request

> 1e-6

every gap the builder checked

A rank is how close the words sit. It is not a judgement of the text.

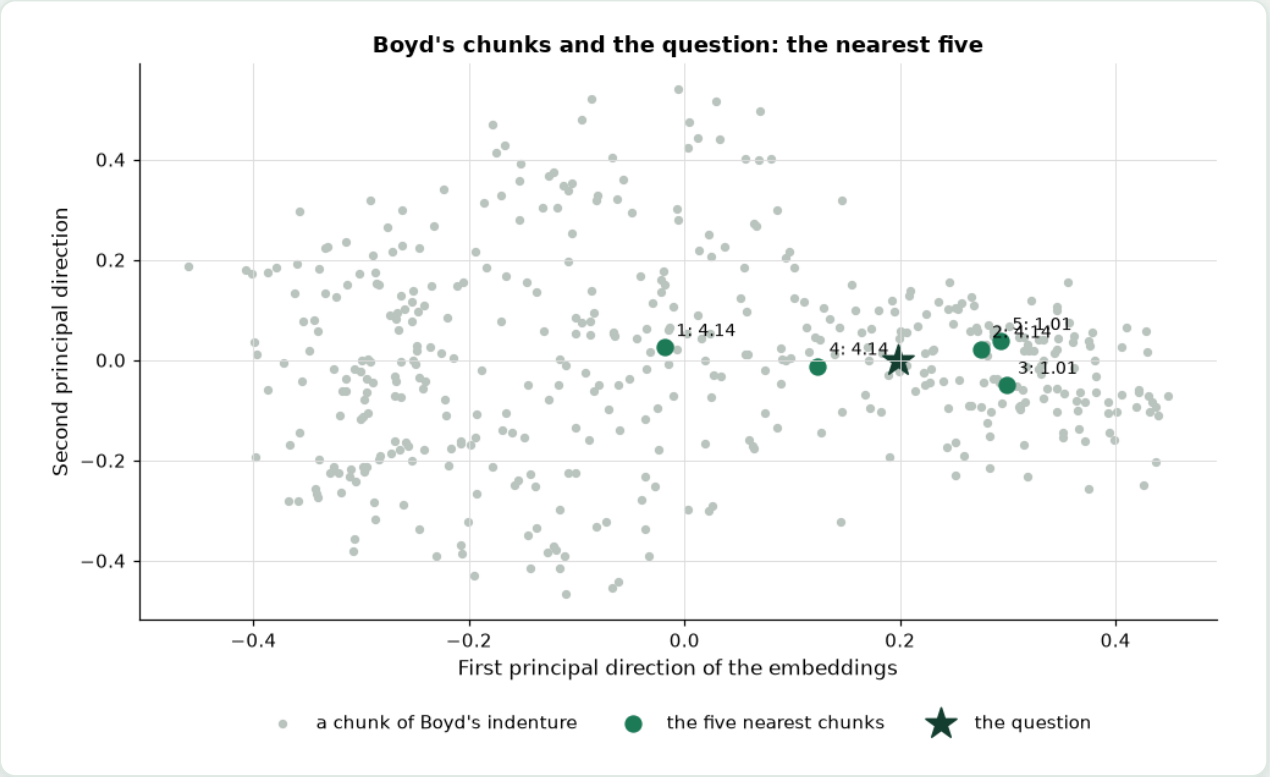
Every setting of the search tool

Setting	Type	Default	What the model reads
`question`	string	required	the question in the indenture's own words, for ex...
k	integer	required	how many chunks, from 1 to 8.

```
characters returned at k = 1, 2, 3, 5, 8: 1,233, 2,489, 3,796, 6,458, 10,346
```

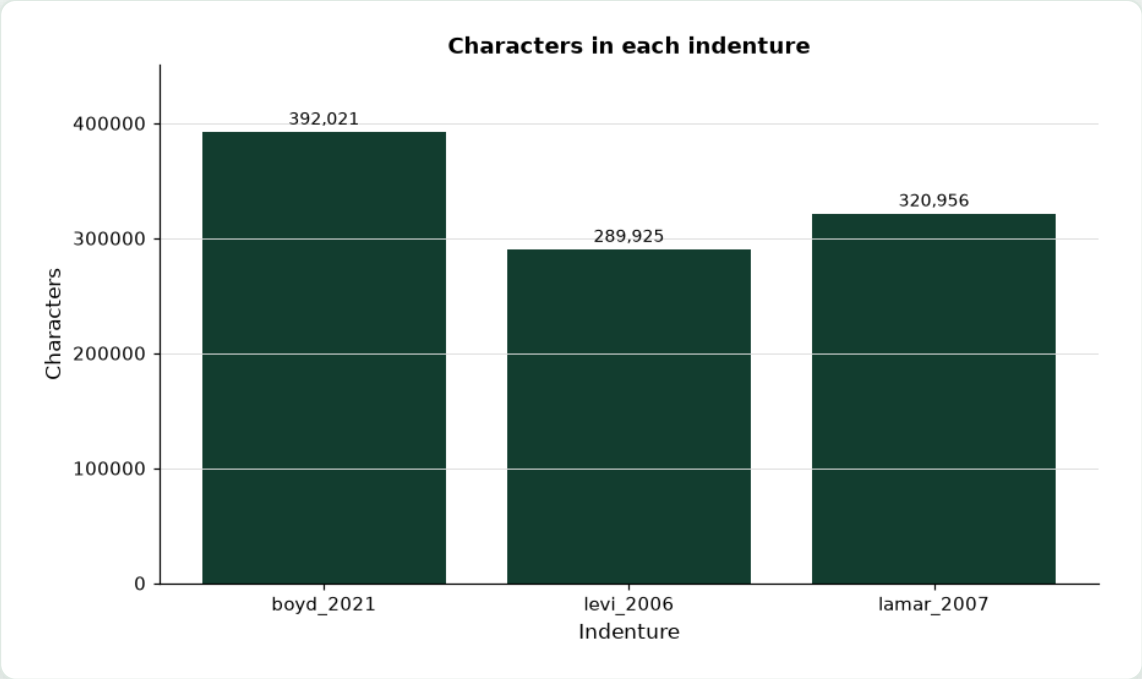
Every hit is a whole chunk with its source line, so the text grows with k. Three is enough for one idea.

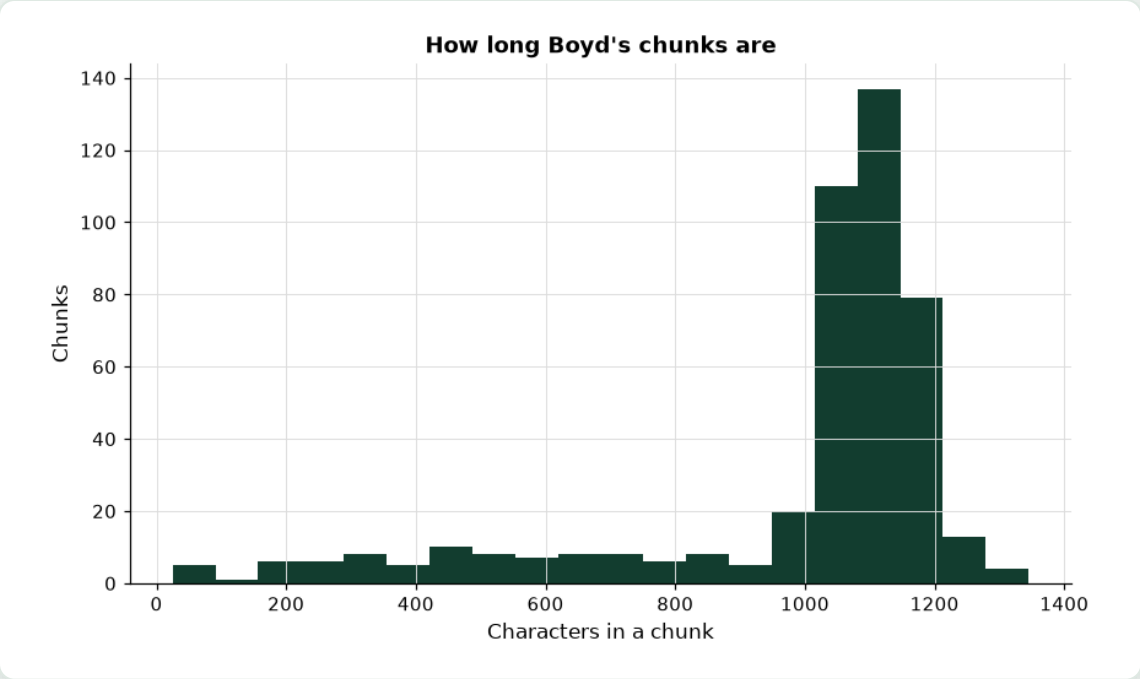
Boyd's chunks and the question



A view of 384 numbers in two: a chunk can look near in the picture and be far in the search.

The documents, drawn





Left: the three indentures. Right: the most text one search hit can add.

Both charts describe the files. A tool returns a slice, never a whole document.

How the two checks work

The problem	A quote and a section number look grounded whether or not they are
citations_ok	Every number and every cited section must appear in what the run's tools returned
quotes_found	Every passage in quotes must appear in that text
You control	The answer's form: the prompt asks for a quote and a section
What comes out	An accepted flag with what is missing, and a found flag for each quote

Quotes and citations, checked

citations_ok (Course 7's judge) and quotes_found, on the saved answers

run	accepted	citations	numbers	missing	quotes found
fomc	False	False	False	no section read [True]	
section	True	True	True	[]	[False]
search	True	True	True	[]	[True]

moved to [Section 4.10]: {'numbers_ok': True, 'citations_ok': False, 'accepted': False, 'missing': []}

"the Company's Consolidated Fixed Charge Coverage Ratio would have been at least 2.0 to 1.0, after gi...

section: the quote wrote 'Pro Forma Basis' in single quotes; the section has double. A changed quote mark is a changed quote.

NOT IN TEXT, through a tool

What the agent did, step by step

```
1  read_section  {"indenture":"boyd_2021","number":"4.12","max_chars":6000} tool_calls 768 31
```

The saved answer

```
NOT IN TEXT

citations_ok False: no section cited; numbers_ok True
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 3,002 prompt and 35 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

The reply the prompt asks for. The judge never accepts NOT IN TEXT: count those replies apart.

An 8-K section, described

Item 2.03, as its filer wrote it: the saved answer's quote

```
The section states: "On June 15, 2021, Public Service Company of New Hampshire, doing business as...

$350,000,000 aggregate principal amount of its 2.20% First Mortgage Bonds, Series V, due 2031
```

The answer wrote the inner quotes as \"Company\"; the quote check reads them as plain quotes.

```
quotes found [True, True]; numbers no tool returned []
```

Source: PUBLIC SERVICE CO OF NEW HAMPSHIRE, Form 8-K filed 2021-06-21, accession 0001104659-21-082904, Item 2.03.

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 2,013 prompt and 237 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

What the filer wrote, under the item it chose. Not a view on the bonds, not a view on the issuer.

How a long result costs

The problem	A model has no memory between requests
The transcript	Each request carries the system prompt, the question, every tool request, every result
The bill	A result is paid for when it arrives and again at every later request
You control	max_chars, which cuts a result, and the step limit, which cuts the later requests
What comes out	A prompt-token count at each request: it steps up by about each result

What a long tool result costs

Prompt tokens by request, in the saved runs of the lesson

run	tokens by request	added	tool result characters
fomc	768, 1061	293	1109
section	789, 2255	1466	6368
search	789, 1173, 1411	384, 238	1372, 639
yield	768, 2234	1466	6368
eightk	789, 1224	435	1377

1466

section run: 4.12 at max_chars 6000

1466

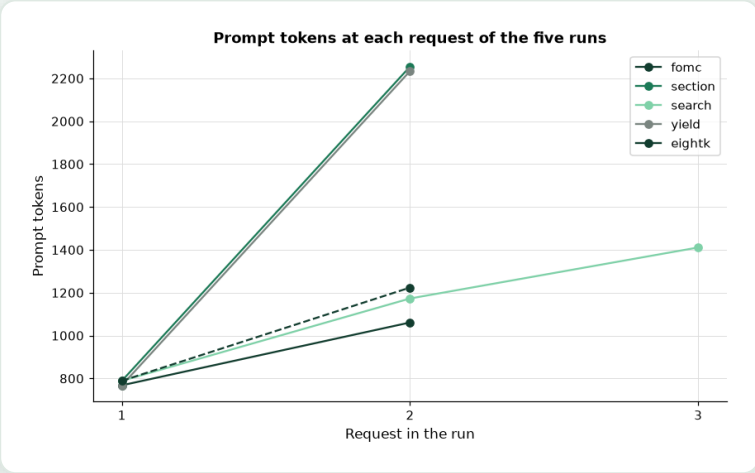
yield run: 4.12 at max_chars 6000

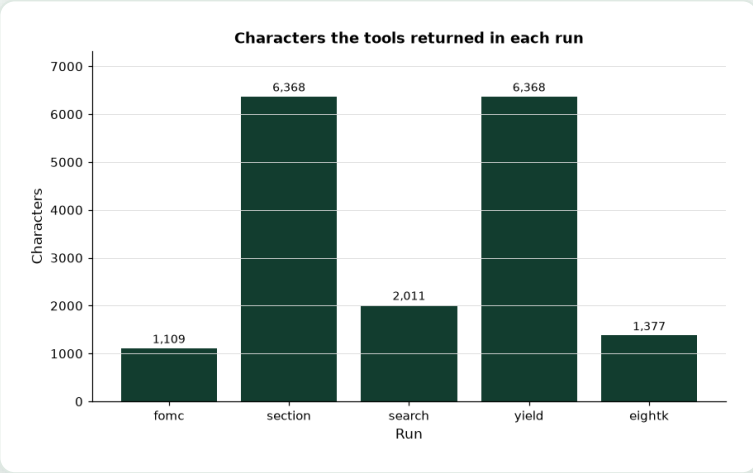
238

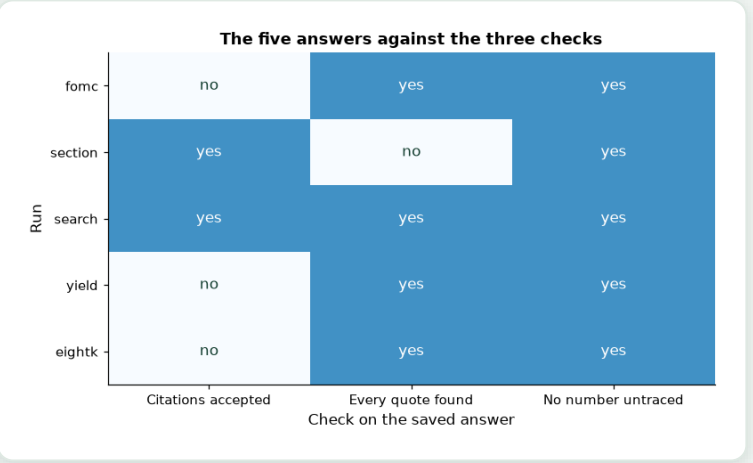
search run: 4.14 at max_chars 300

Every later request carries the whole transcript: a long result is paid for again at every step.

The five runs, drawn







Left: a long result is resent at every later request. Right: the checks on each answer.

A light cell is a check the saved answer did not meet; the run's own check lines say which.

Excel and Git

Excel side by side (tested in Excel)

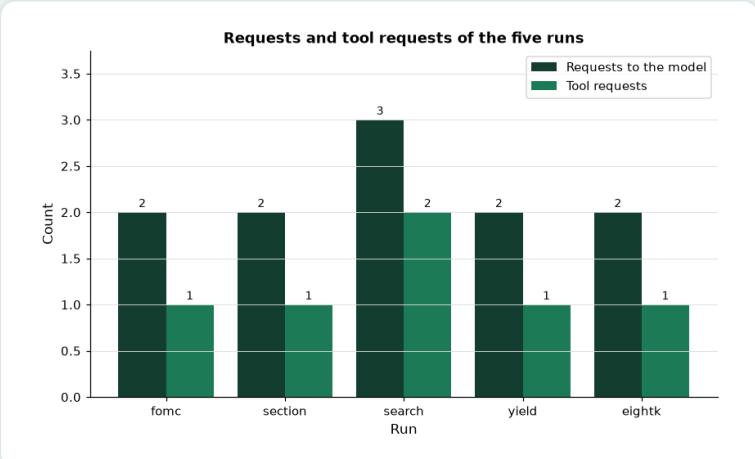
`=ISNUMBER(MATCH(B2,C2:G2,0))`	`B 4.12 (text)`	`C 4.12`	`(section)`	TRUE
`=ISNUMBER(MATCH(B3,C3:G3,0))`	`B 4.14 (text)`	`C 4.14`	`(search)`	TRUE
`=ISNUMBER(MATCH(B5,C5:G5,0))`	`B 4.12 typed as a number`			FALSE
`=C8-C7`	`prompt tokens, section run`			1466
`=C11-C10`	`prompt tokens, yield run`			1466

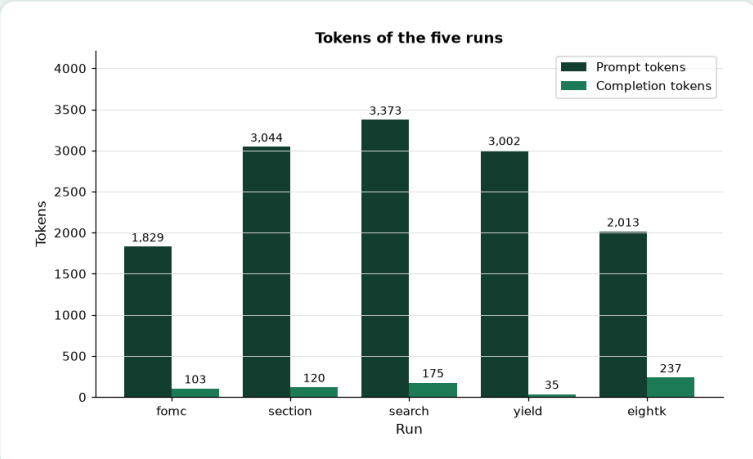
Excel against Python: 4 of 4 rows the same

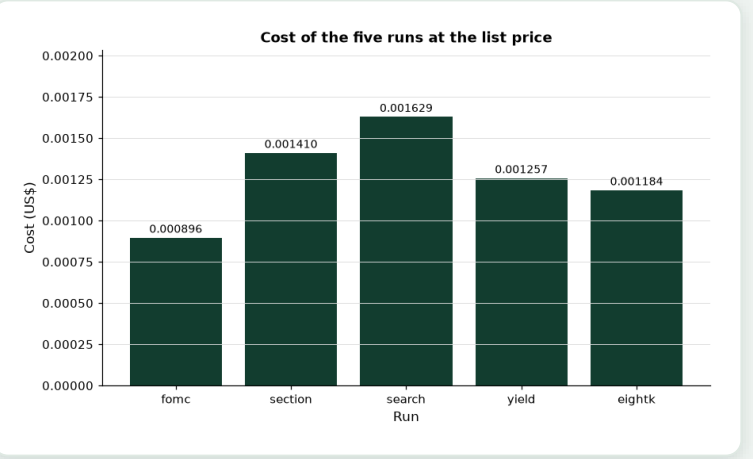
```
...at least 100, for example [-6000.-]{+2000.+}
first request's key: 6f5f45311685 saved, 32ceba4ebfca edited
```

git diff --word-diff: one word of a tool description, and every request key changes. 42 passed.

What each run used and cost







Run	Requests	Tool requests	Prompt	Completion	Cost (US$)	Passed
fomc	2	1	1,829	103	0.000896	True
section	2	1	3,044	120	0.001410	True
search	3	2	3,373	175	0.001629	True
yield	2	1	3,002	35	0.001257	True
eightk	2	1	2,013	237	0.001184	True

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Read the section from the body: its first sentence, never the contents line.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

The text for 4.12 must hold Incur any Indebtedness. Run it: it fails.

→

4

Fix

Let textkit.find_sections find it: the last heading, the body's.

Today: text.find returns the first heading, the table of contents line, so the reader returns no covenant.

Code that runs is not the same as code that is right.

What you can do now

- Documents as tools, every text with its source line
- Read a section from the body, not the contents
- Search as a tool: ranks and texts, never scores
- citations_ok and quotes_found; read the missing list
- NOT IN TEXT through a tool; an 8-K described

Week 2, Day 3 MCP: the same tools, served by a server